LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Branch: CSM
Section: B
Task No.: 5
Student Name: K. Saranya
Roll No.: A24126552090

1. TITLE

Develop a Dynamic Student Registration Webpage Using DOM & JavaScript.

2. AIM

To develop a dynamic Student Registration webpage using HTML, CSS, and JavaScript with DOM manipulation that allows users to add, display, and delete student records dynamically.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of the Document Object Model (DOM).
- To access HTML elements using JavaScript.
- To handle the form submit event using an event listener.
- To prevent the default page-reload behaviour of a form.
- To dynamically create and modify HTML table rows and cells.
- To add and delete student records dynamically.
- To perform input validation using JavaScript.
- To understand how JavaScript updates webpage content without reloading the page.

4. THEORY

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. JavaScript uses the DOM to access, modify, add, or remove elements and content dynamically.

In this experiment, a Student Registration webpage is developed using HTML, CSS, and JavaScript. The HTML file provides the basic structure, CSS provides the visual appearance, and JavaScript handles the dynamic behaviour of adding and deleting student rows.

JavaScript accesses HTML elements using document.getElementById(), and listens for the form's submit event using addEventListener(). The application uses DOM methods such as:

- getElementById() — selects an HTML element using its id.
- createElement() — creates a new HTML element dynamically, such as a table row or cell.
- appendChild() — adds a newly created element as a child of an existing element.
- textContent — sets the visible text inside an element.

- className — assigns a CSS class to an element, used here to style the Delete button.
- remove() — removes an element from the DOM, used to delete a student row.
- event.preventDefault() — stops the form from reloading the page on submit.

The basic working flow is:

User Input → Form Submit Event → Validation → DOM Manipulation → Update Student Table → Display Message

5. ALGORITHM / PROCEDURE

1. Create a project folder for the Student Registration application.
2. Create an HTML file named index.html.
3. Create a CSS file named style.css.
4. Create a JavaScript file named script.js.
5. Design the webpage using a form with Student Name, Roll Number, and Course fields.
6. Add a Course dropdown with predefined subject options.
7. Add an Add Student submit button.
8. Create a table with a header row (Name, Roll Number, Course, Action) to display students.
9. Access the required HTML elements using document.getElementById().
10. Add a submit event listener to the form and call event.preventDefault() to stop the page from reloading.
11. Validate that the Name, Roll Number, and Course fields are not empty.
12. Create a new table row and table cells using document.createElement().
13. Set the cell contents using textContent.
14. Create a Delete button for the row and assign it the deleteBtn class.
15. Add a click event listener to the Delete button to remove its row using remove().
16. Append the cells to the row and the row to the student table body.
17. Display a success or validation message and clear the input fields.
18. Execute the webpage in a browser and verify the dynamic operations.

6. PROGRAM

index.html

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1 id="title">Student Registration Form</h1>
        <form id="studentForm">
            <label>Student Name:</label>
            <input type="text" id="name" placeholder="Enter student name">
            <label>Roll Number:</label>
            <input type="number" id="roll" placeholder="Enter roll number">
            <label>Course:</label>
```

```
            <select id="course">
                <option value="">Select Course</option>
                <option value="JavaScript">JavaScript</option>
                <option value="Python">Python</option>
                <option value="Java">Java</option>
            </select>
            <button type="submit">Add Student</button>
        </form>
        <p id="message"></p>
        <h2>Student List</h2>
        <table id="studentTable">
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Roll Number</th>
                    <th>Course</th>
                    <th>Action</th>
                </tr>
            </thead>
            <tbody id="studentList">
            </tbody>
        </table>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

style.css

style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
}
.container {
    width: 700px;
    margin: 50px auto;
    background-color: white;
    padding: 30px;
    border-radius: 10px;
}
h1 {
    text-align: center;
}
label {
    display: block;
    margin-top: 10px;
}
input,
select {
    width: 100%;
    padding: 10px;
    margin-top: 5px;
    box-sizing: border-box;
}
button {
    margin-top: 15px;
    padding: 10px 20px;
```

```
    cursor: pointer;
}
table {
    width: 100%;
    margin-top: 20px;
    border-collapse: collapse;
}
th,
td {
    border: 1px solid black;
    padding: 10px;
    text-align: center;
}
th {
    background-color: #ddd;
}
.deleteBtn {
    background-color: red;
    color: white;
    border: none;
}
```

script.js

script.js

```
// Getting HTML elements using DOM
let form = document.getElementById("studentForm");
let nameInput = document.getElementById("name");
let rollInput = document.getElementById("roll");
let courseInput = document.getElementById("course");
let studentList = document.getElementById("studentList");
let message = document.getElementById("message");

// Form submit event
form.addEventListener("submit", function(event) {
    // Prevent page refresh
    event.preventDefault();

    // Getting values from input fields
    let name = nameInput.value;
    let roll = rollInput.value;
    let course = courseInput.value;

    // Checking empty fields
    if (name === "" || roll === "" || course === "") {
        message.textContent = "Please fill all fields";
        return;
    }

    // Creating a new table row
    let row = document.createElement("tr");

    // Creating table cells
    let nameCell = document.createElement("td");
    let rollCell = document.createElement("td");
    let courseCell = document.createElement("td");
    let actionCell = document.createElement("td");
```

```
33
34      // Adding values to cells
35      nameCell.textContent = name;
36      rollCell.textContent = roll;
37      courseCell.textContent = course;
38
39      // Creating Delete button
40      let deleteButton = document.createElement("button");
41      deleteButton.textContent = "Delete";
42      deleteButton.className = "deleteBtn";
43
44      // Delete button event
45      deleteButton.addEventListener("click", function() {
46          row.remove();
47          message.textContent = "Student deleted successfully";
48      });
49
50      // Adding delete button to action cell
51      actionCell.appendChild(deleteButton);
52
53      // Adding cells to row
54      row.appendChild(nameCell);
55      row.appendChild(rollCell);
56      row.appendChild(courseCell);
57      row.appendChild(actionCell);
58
59      // Adding row to table
60      studentList.appendChild(row);
61
62      // Display success message
63      message.textContent = "Student added successfully";
64
65      // Clear input fields
66      nameInput.value = "";
67      rollInput.value = "";
68      courseInput.value = "";
69  });
```

7. CODE EXPLANATION

Code / Concept	Explanation
document.getElementById()	Selects HTML elements using their unique ids.
form.addEventListener("submit")	Runs a function whenever the registration form is submitted.
event.preventDefault()	Stops the form's default action of reloading the page.
.value	Gets the value entered in an input or selected in a dropdown.
if	Validates that the name, roll number, and course fields are not empty.
message.textContent	Displays a validation or success message to the user.
document.createElement()	Creates a new HTML element, such as a table row or cell.
textContent	Sets the visible text inside a newly created cell.
className	Assigns the deleteBtn CSS class to the Delete button.

Code / Concept	Explanation
appendChild()	Adds a created cell or row as a child of another element.
row.remove()	Removes the selected student's row from the table.
nameInput.value = ""	Clears the input fields after a student is added.

8. TEST CASES AND EXECUTION DATA

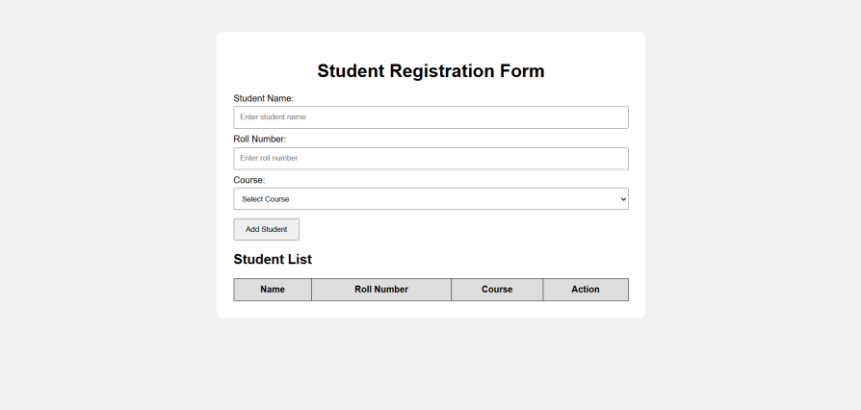
The Student Registration webpage was opened in a browser and tested for its initial state, form entry, and successful submission; the results were recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	The Student Registration Form and Student List table should load correctly	Page displayed correctly, as shown in Output 1	Pass
TC-02	Check the Student Name, Roll Number, and Course fields on load	All fields should display their placeholder text and remain empty until the user types	Fields displayed empty with correct placeholders, as shown in Output 1	Pass
TC-03	Check the Student List table before any student is added	The table header (Name, Roll Number, Course, Action) should be visible with an empty list body	Table header displayed correctly with no data rows, as shown in Output 1	Pass
TC-04	Enter K.Saranya, 90, and select JavaScript in the form fields	The entered values should appear correctly in their respective fields	Values displayed correctly in all three fields, as shown in Output 2	Pass
TC-05	Click 'Add Student' after filling all fields	The student should be added to the table and a success message should be shown	"Student added successfully" message shown and new row displayed, as shown in Output 3	Pass
TC-06	Check the Student List table after submission	The new row should display the correct Name, Roll Number, Course, and a Delete button	Row displayed as K.Saranya, 90, JavaScript, with a Delete button, as shown in Output 3	Pass

9. OUTPUT

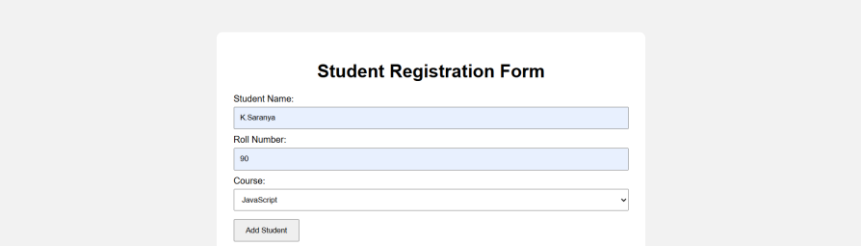
Output 1: Student Registration Form — Initial Empty State

The following screenshot shows the Student Registration Form on initial page load. It displays the registration form with the Student Name, Roll Number, and Course fields, the Add Student button, and the empty Student List table.



Output 2: Student Registration Form — Fields Filled Before Submission

The following screenshot shows the form after entering a student's details — Student Name: K.Saranya, Roll Number: 90, and Course: JavaScript — just before clicking Add Student.



Output 3: Student Registration Form — After Successful Submission

The following screenshot shows the page after clicking Add Student. The success message "Student added successfully" is displayed, and the new student row (K.Saranya, 90, JavaScript) appears in the Student List table with a Delete button.

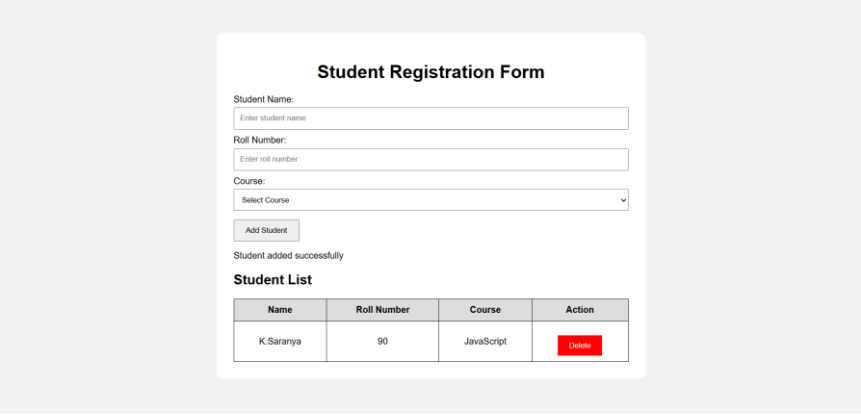


Figure 1: Output of Dynamic Student Registration Form Using DOM and JavaScript

10. RESULT

Thus, a dynamic Student Registration webpage was successfully developed using HTML, CSS, and JavaScript with DOM manipulation. The application allows users to add and delete student records dynamically, with input validation and event handling implemented using JavaScript. The webpage was executed and verified in a web browser for its initial state, form entry, and successful submission, and the expected outputs were obtained.